Title Page

Maily Detector

Group 10

Introduction

Email is no longer just a contact tool it has become an essential part of our daily lives. From subscriptions , job applications and important notifications. knowing every mail we received become challenging.

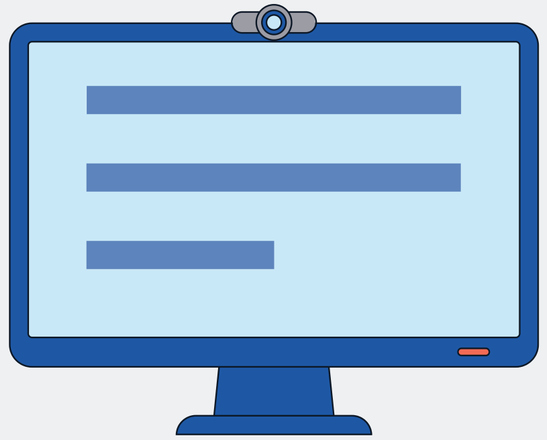
This is where our “Maily detector” comes in. Our app helps users organize their inbox by identifying subscription emails and detecting scam mail.

Making managing your inbox easier, safer, and more efficient.



Design Consideration

Assumptions



- The model processes plain text only
- The model was trained on English-language data only
- The user accesses email via Gmail on Google Chrome
- The Hugging Face Space is running and awake at the time of use

Insert your topic here

Design Consideration

Technology Constraints

- Chrome MV3
- DistilBERT token limit
- Hugging Face free tier
- Gradio API versioning









Design Consideration

System Environment

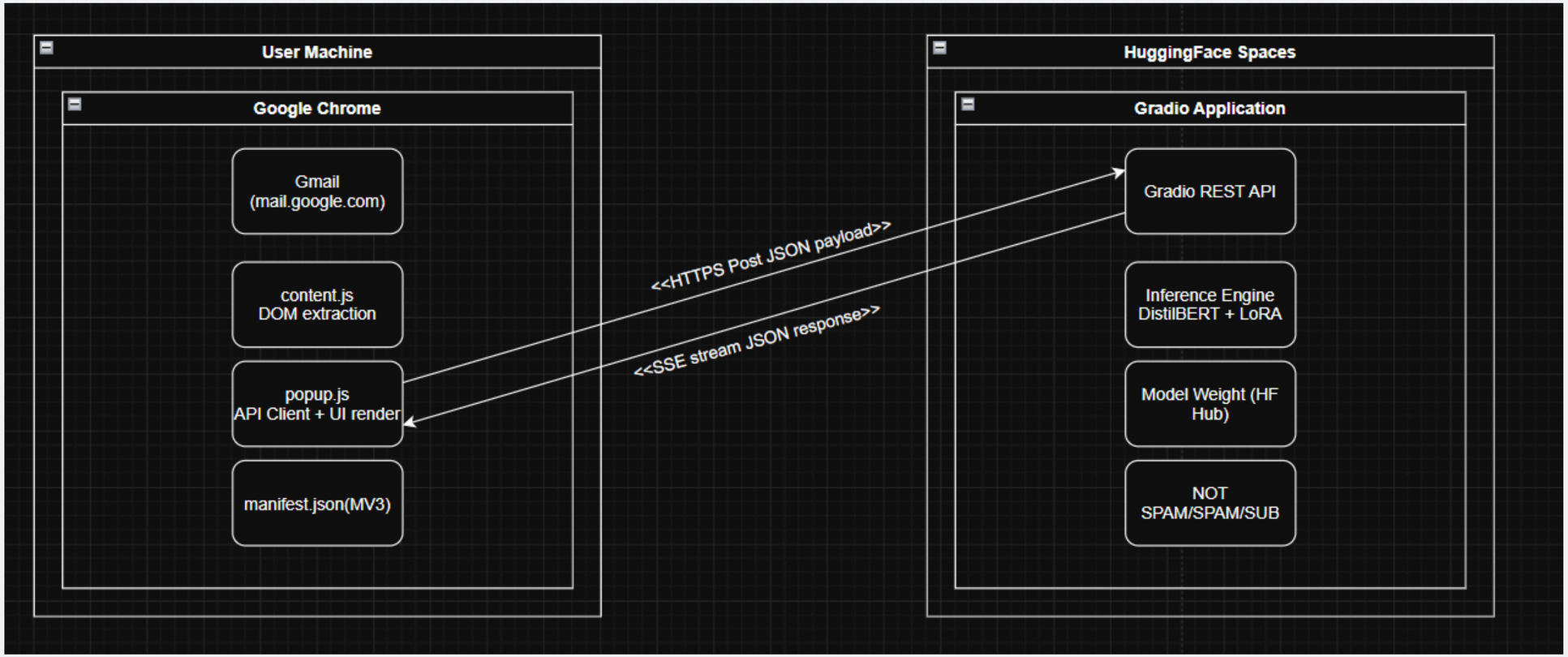
Node 1: User Machine (Chrome Browser)

- Gmail running at mail.google.com
- Chrome Extension (MV3)
  - content.js — injected into Gmail, extracts email text from DOM
  - popup.js — sends text to HF Space API, renders result

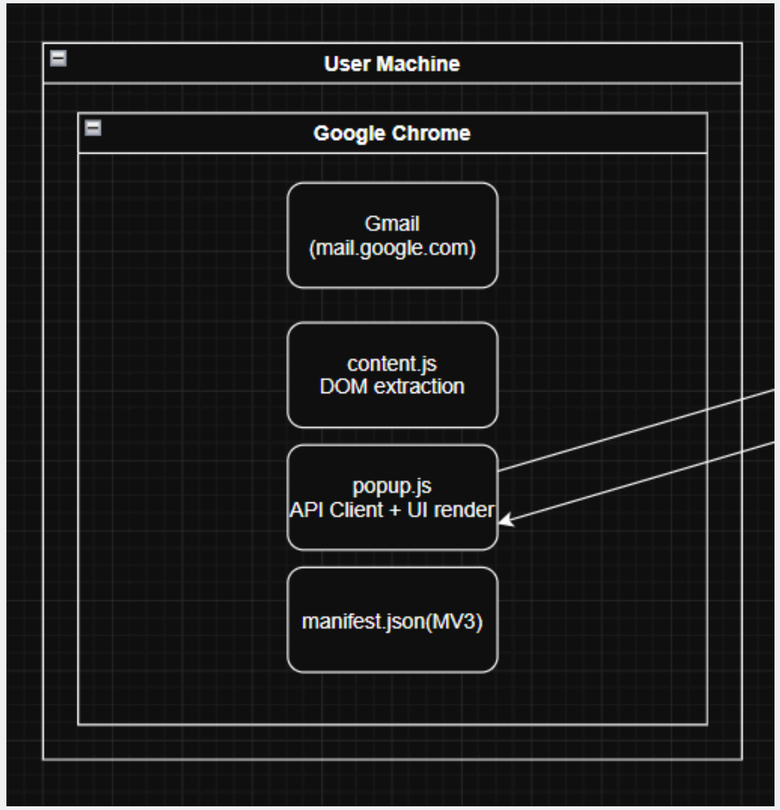
Node 2: Hugging Face Spaces (Cloud)

- Gradio App
  - Gradio REST API (/gradio_api/call/predict)
  - DistilBERT + LoRA inference engine (3-class classifier)
  - Model weights stored in HF Hub

Architecture



Architecture

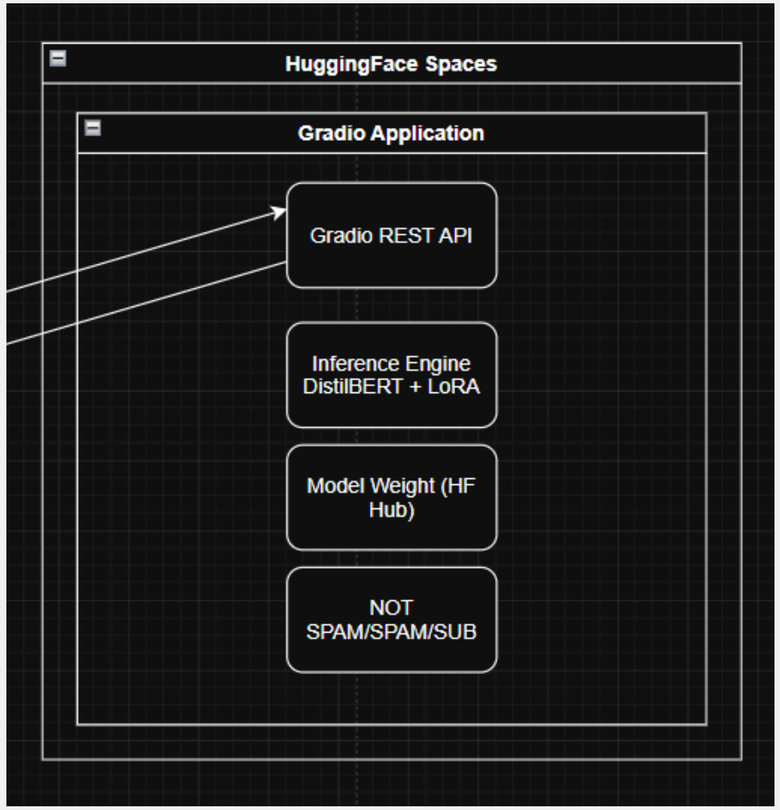


The user's machine runs Google Chrome with the Maily Detector extension installed. Inside Chrome, there are two active components.

1. **content.js** is a content script that gets automatically injected into any Gmail page the user opens.
2. **popup.js** runs inside the extension popup when the user clicks the extension icon.

- System sends a message to content.js to request the email text, then takes that text and fires an API call to the HuggingFace Space, and finally renders the result back to the user in the popup UI.

Architecture



The cloud node hosts the Gradio application at spammail-detector.hf.space. Inside this node, there are three layers.

1. **Gradio REST API** acts as the entry point. It receives the incoming request from the extension and routes it to the inference engine.
2. **Inference engine** is where the actual AI classification happens. The model reads the email text, processes it, and produces one of three labels: **NOT SPAM**, **SPAM**, or **SUBSCRIPTION**
3. **Model weights** are stored in HuggingFace Hub, which the inference engine loads when the Space starts up.

- The DistilBERT model classifies the email. The result travels back to popup.js over HTTPS. The pop-up displays the label, confidence percentage, and details to the user.

DistilBERT Model info

Model Info:

- Base model: DistilBERT
- Fine-tuned with LoRA
- 3-class sequence classification

Dataset:

- https://www.kaggle.com/datasets/jackksoncsie/spam-email-dataset
- https://www.kaggle.com/datasets/mirzayasirabdullah07/subscription-renewal-and-cancellation-dataset
- https://www.kaggle.com/datasets/ganiyuolalekan/spam-assassin-email-classification-dataset

Deployment:

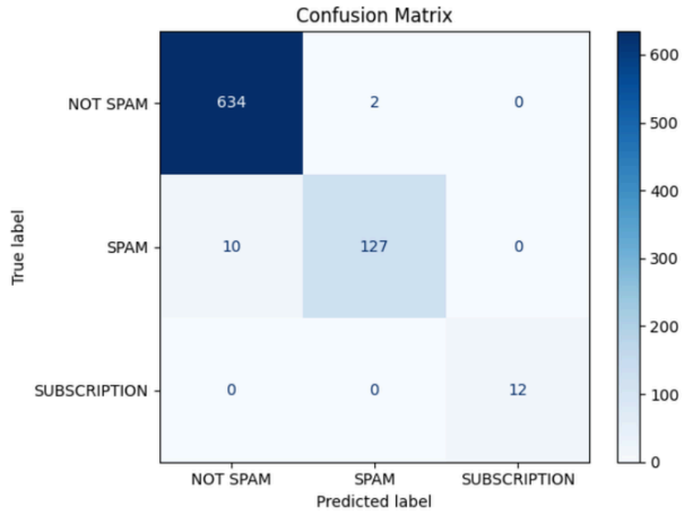
- Hugging face space
- Live API endpoint
- No sever maintainence needed

Model Training result

```
...                precision    recall  f1-score   support

    NOT SPAM       0.98      1.00      0.99       636
        SPAM       0.98      0.93      0.95       137
SUBSCRIPTION       1.00      1.00      1.00        12

    accuracy                           0.98       785
   macro avg       0.99      0.97      0.98       785
weighted avg       0.98      0.98      0.98       785
```




```
Confusion matrix saved as confusion_matrix.png
```

Training Score Meanings:

Precision: Of the emails predicted as “phishing”, how many are actually phishing?

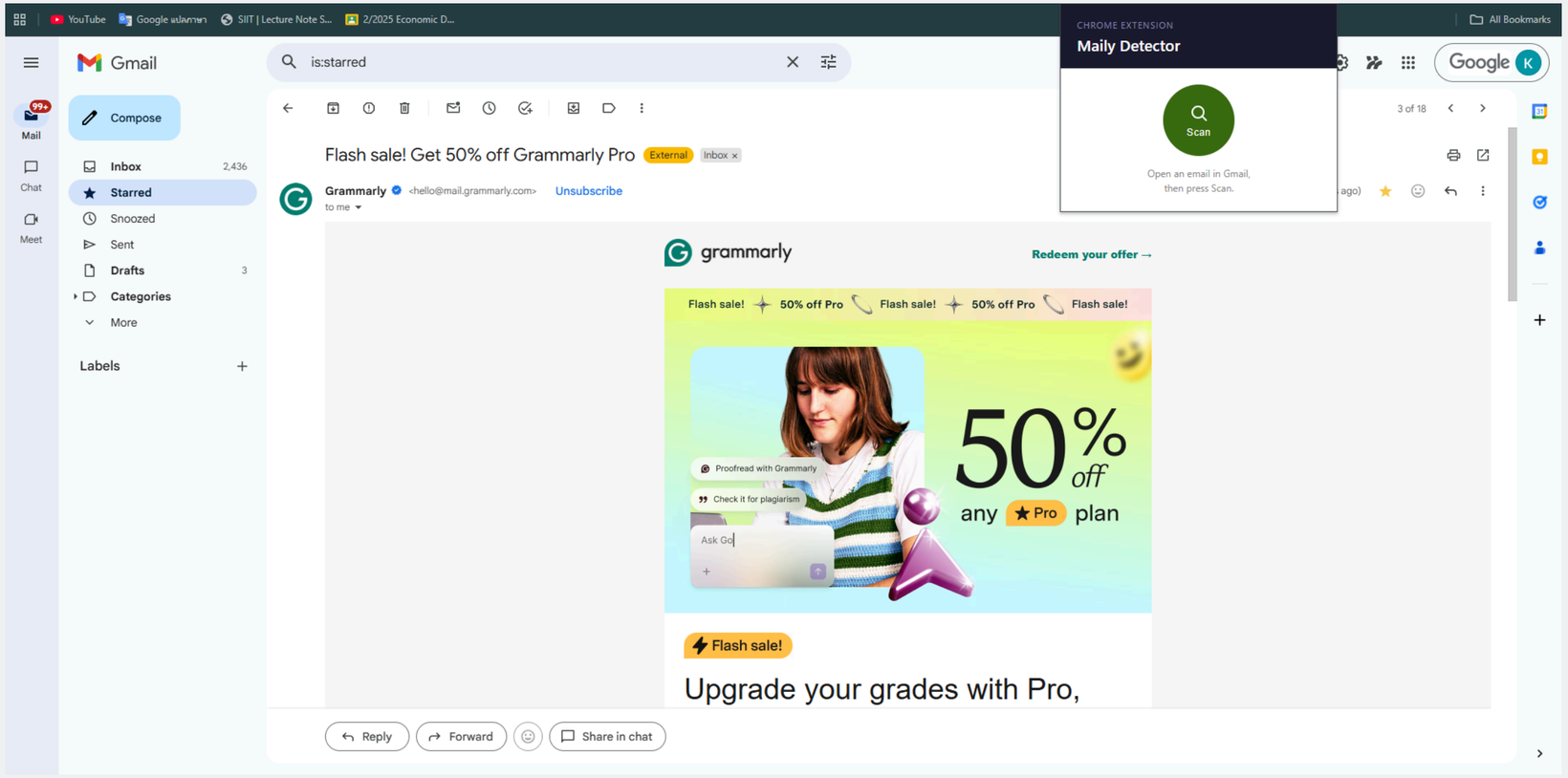
Recall: Of all actual phishing emails, how many did the model catch?

F1-score: Balance between precision and recall. Overall score

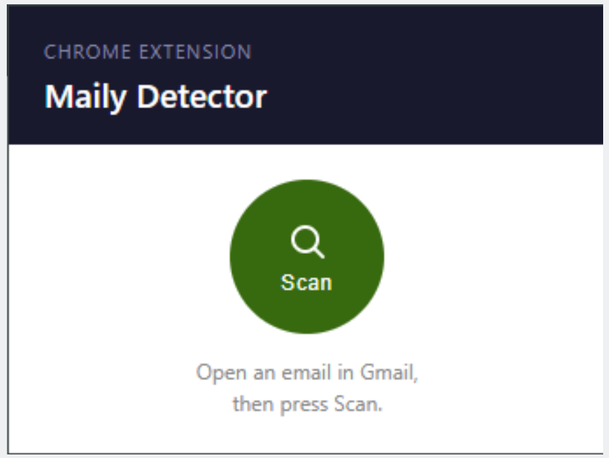
Support: Number of actual samples

Insert your topic here

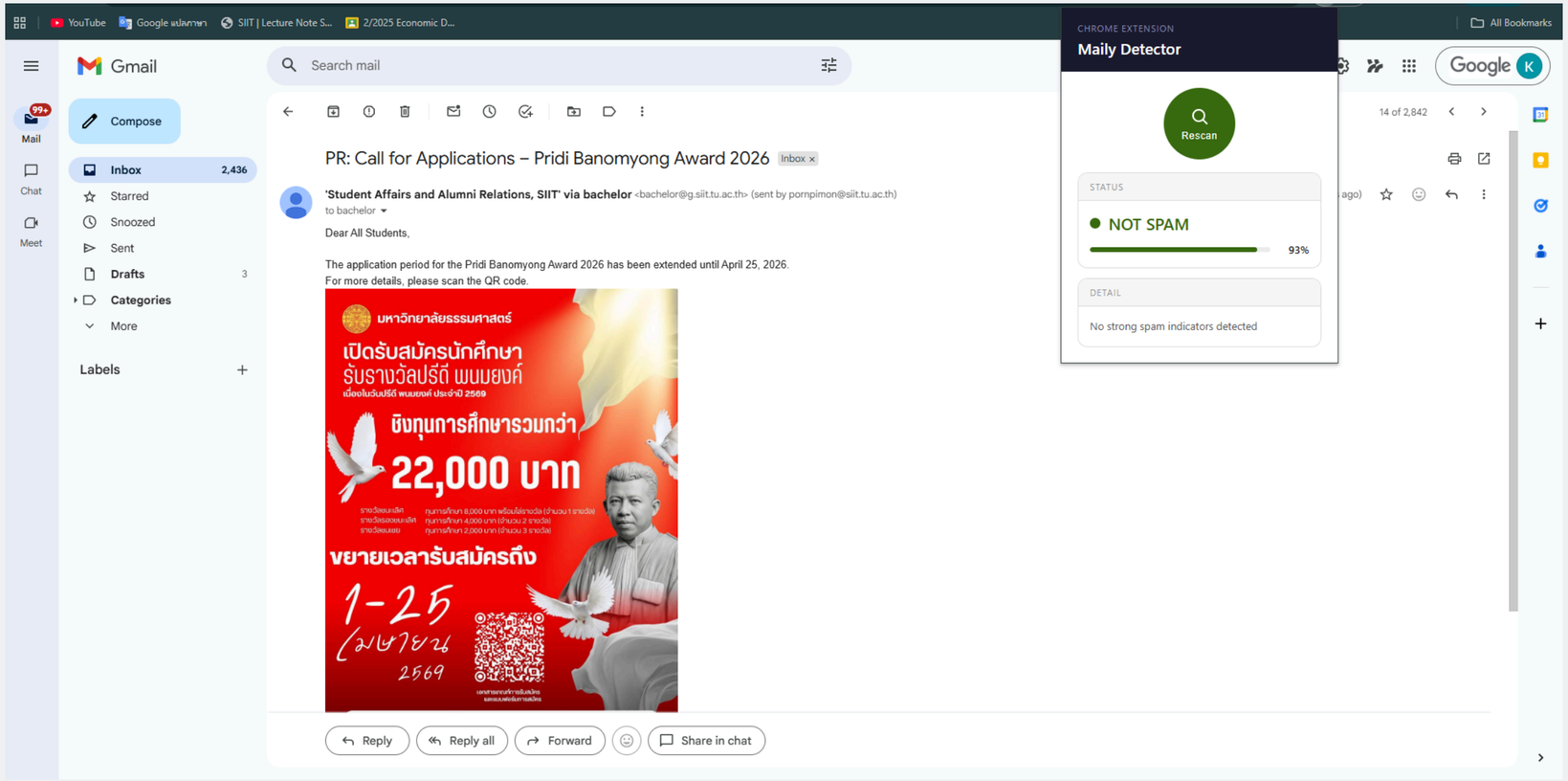
UX/UI Design



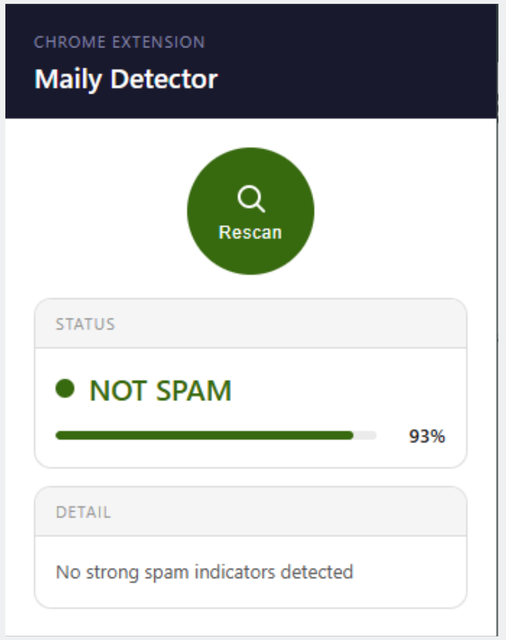
- Initial state
  - extension popup before scanning



UX/UI Design

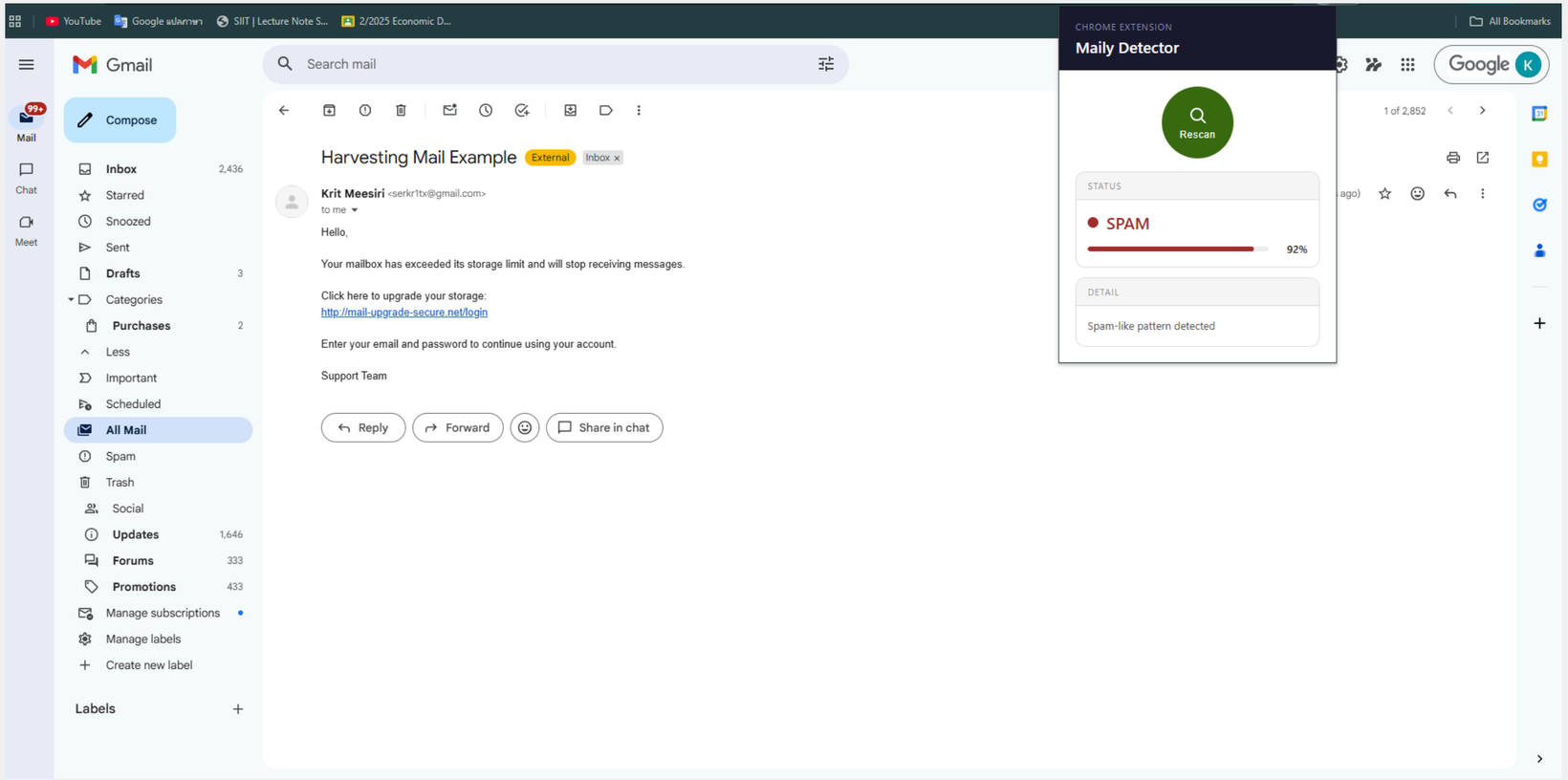


- **Result state (NOT SPAM)**
  - shows green status, confidence bar, and reason

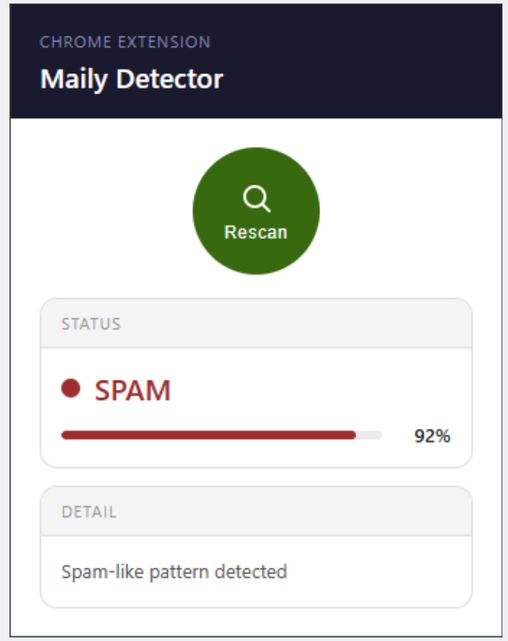


Insert your topic here

UX/UI Design

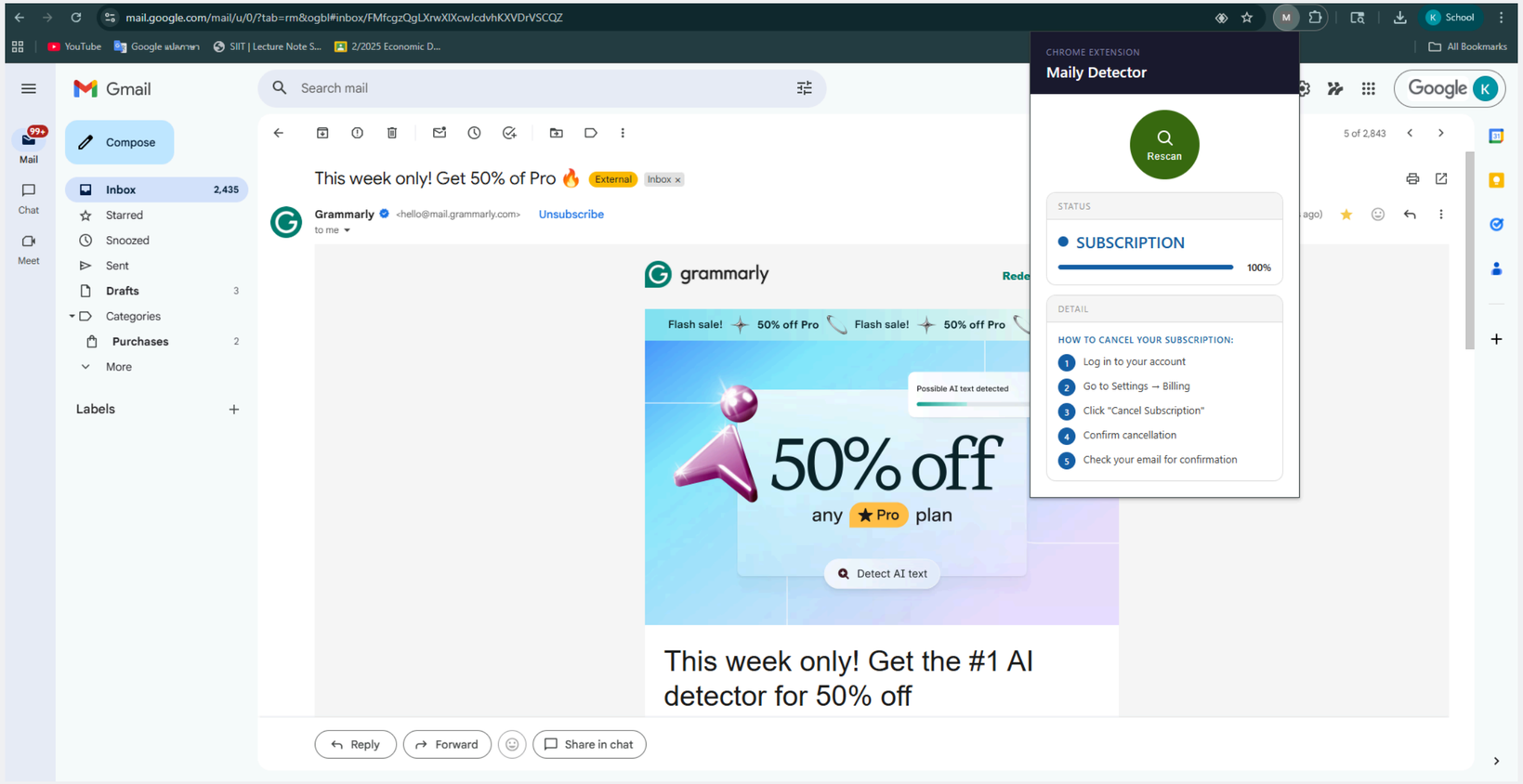


- **Result state (SPAM)**
  - shows red status with spam keywords detected

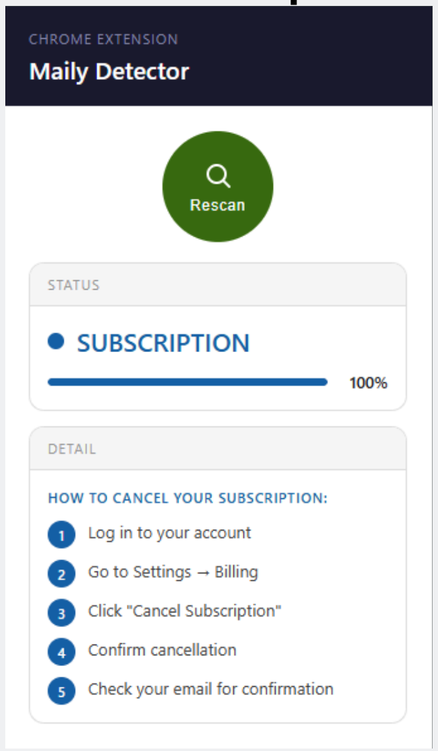


Insert your topic here

UX/UI Design

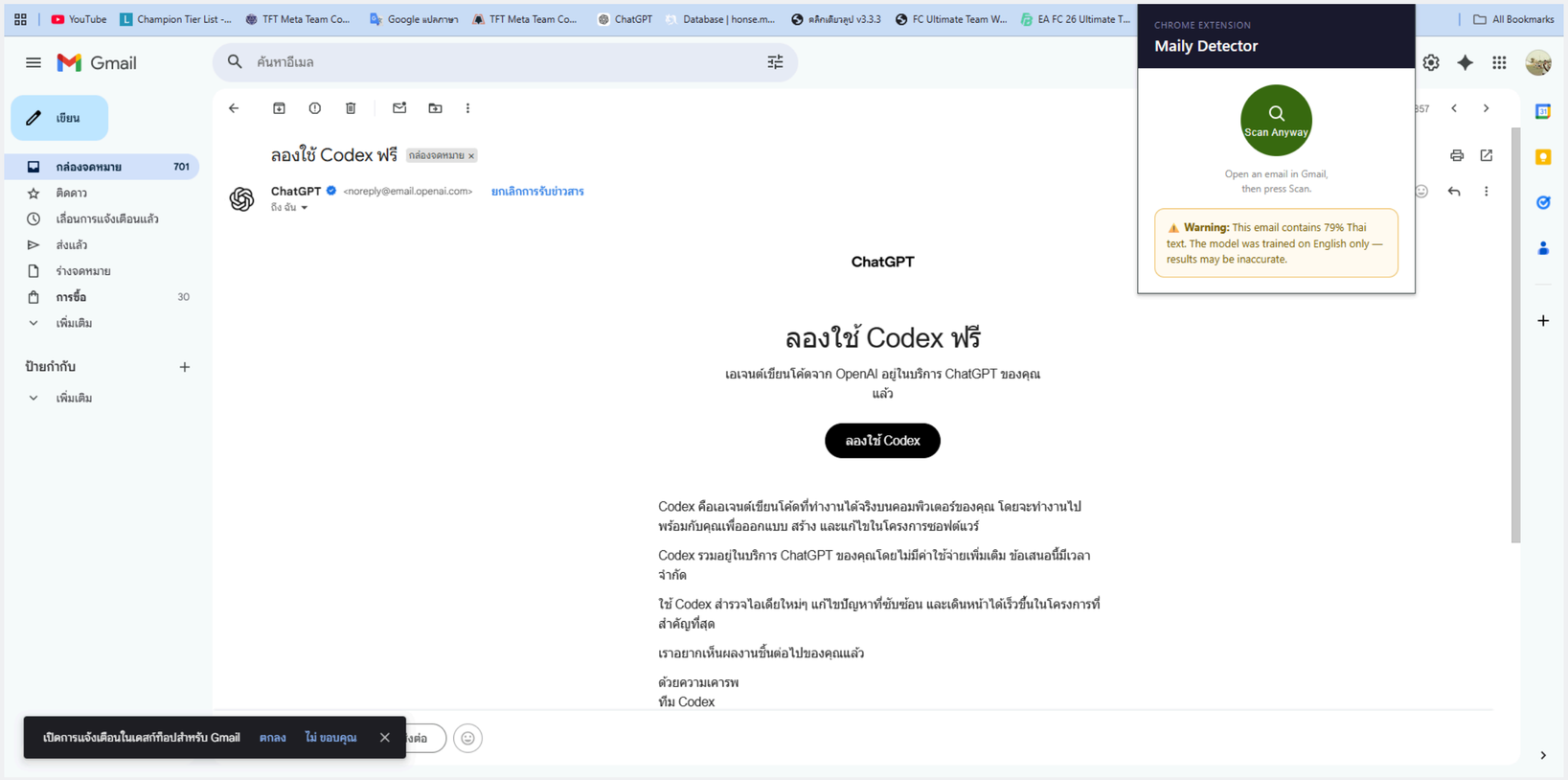


- **Result state (SUBSCRIPTION)**
  - shows **blue status** with canceled steps

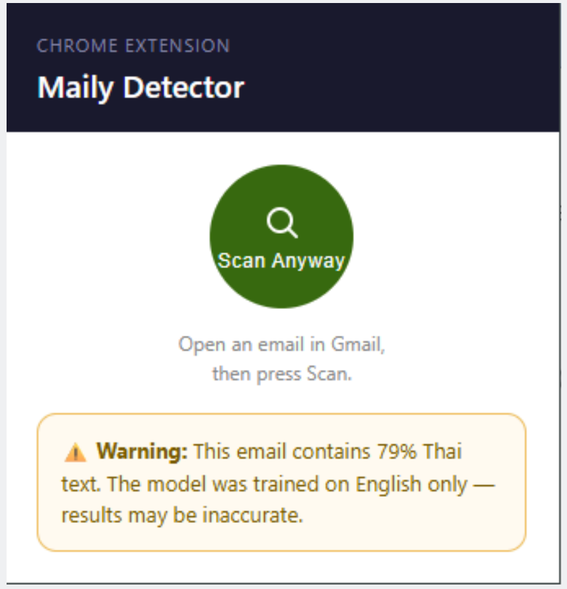


Insert your topic here

UX/UI Design

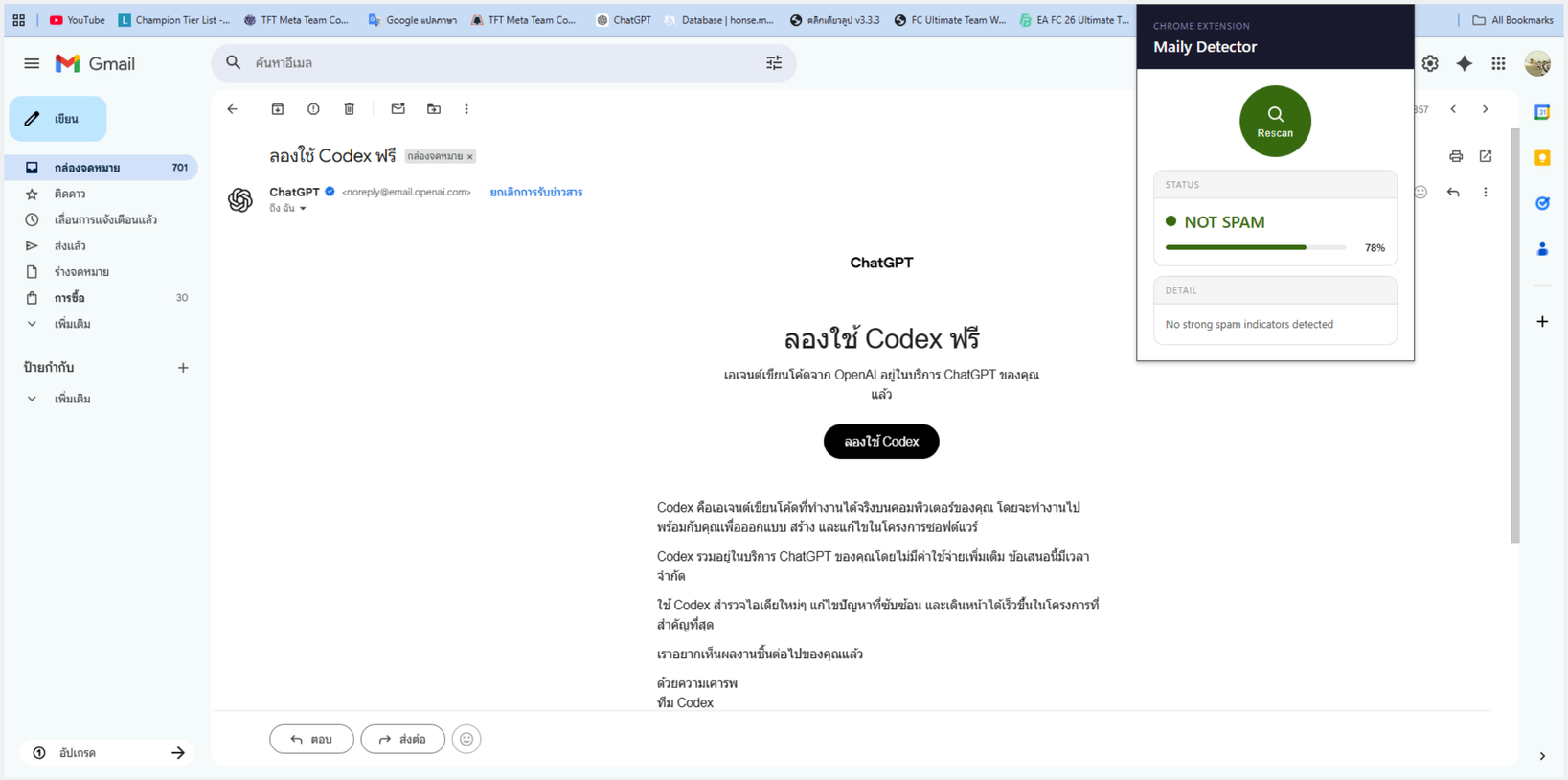


- **Warning state**
  - **Thai language detected**

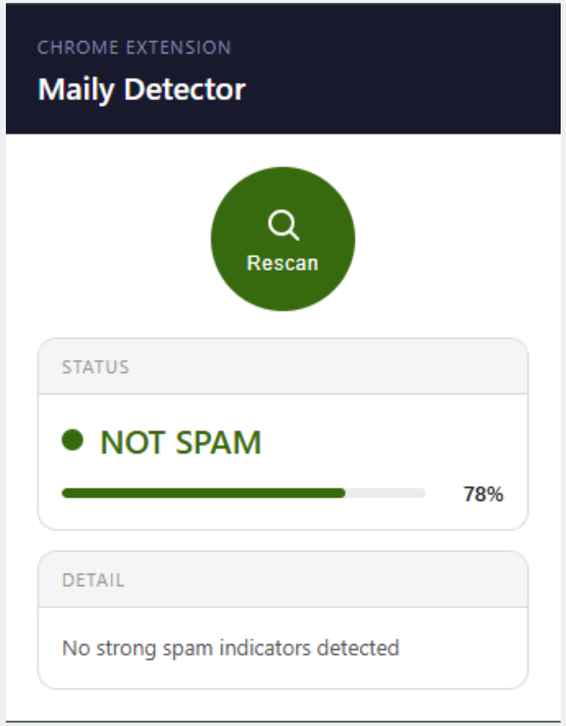


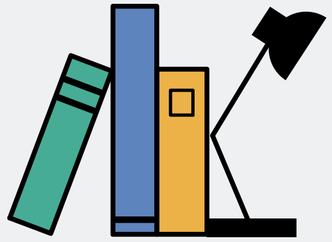
Insert your topic here

UX/UI Design



- **Warning state**
  - **After clicking “Scan Anyway”.**





Thank you!

01 **6622770491 - Apichat Laohawan**

02 **6622780037 - Krit Meesiri**

03 **6622781936 - Apiwit Pengyuak**

04 **6622782009 - Pearanat Pipathanasiri**

05 **6622782033 - Panod Manodee**